Runtime adaptation as a programming pattern in service composition

Carlos G. Lopez Pombo *[1], Pablo Montepagano[2], and Emilio Tuosto[3]

[1] Centro Interdisciplinario de Telecomunicaciones, Electrónica, Computación y Ciencia Aplicada, Universidad Nacional de Río Negro - Sede Andina and CONICET.
[2] Departamento de Computación, Facultad de Ciencias Exactas y Naturales, Universidad de Buenos Aires.
[3] Gran Sasso Science Institute

Abstract. We propose a programming pattern for runtime architectural adaptation of service-oriented applications. Our approach enables transparent reconfiguration of distributed services at runtime leveraging on the key features for dynamic discovery binding of services supported developed in an existing platform. More precisely, in this paper we equip the platform with monitoring capabilities supporting the transparent reconfiguration of the architecture of service-oriented applications triggered by changes in the execution context. We demonstrate the effectiveness of the approach on the adaptable TeaStore benchmark.

1 Introduction

Adaptability can be intuitively understood as the property of a system respond appropriately to changes in its execution environment. Often, adaptation impacts on the behaviour of the system without necessarily affecting its architecture. For instance, fault-tolerance can be tackled by replicating components without affecting the overall architecture. If we embrace the view that for cloud applications this is not necessarily the case, raises the problem of devising adaptation mechanisms of architectures. Arguably, the problem boils down to adopting suitable programming features allowing developers to implement countermeasures against changes in the execution context. Let us introduce a running example that we use here to illustrate this point and we will appeal to in order to demonstrate our approach throughout the paper. Our example elaborates on the adaptable TeaStore benchmark[4] proposed in [2] to evaluate adaptation mechanisms. More precisely, this benchmark is designed to treat adaptability as a first-class design concern in service-oriented architectures. In particular, the benchmark classifies services as mandatory and optional and as external or local, envisages

* On leave from Instituto de Ciencias de la computación CONICET–UBA and Departamento de Computación, Facultad de Ciencias Exactas y Naturales, Universidad de Buenos Aires.

[4] These scenarios are based on the TeaStore use case [1] originally proposed to benchmark micro-services.

multiple versions of components (featuring e.g., different functionalities) in order to specify and evaluate reconfiguration policies across diverse operational scenarios.

Our running example involves a web user-interface, an image provider service, and a locally hosted database service. The web user-interface, say WebUI, is the entry point to the application for users; in particular it displays images of products generated by an image provider, say Image provider. According to the specification in [1],

- image providers manipulate images (e.g., applying filters, resizing, etc.);
- WebUI connects to an image provider service in order to generate webpages to display products to users.

A local storage service, say Local DB, manages a database of images that WebUI can resort to when e.g., the quality of images elaborated by Image provider is sub-optimal. The application can resort to locally stored default images if necessary.

Back to the application-dependent aspects of architectural adaptation, let us consider a few possible ways to realize the reconfiguration required when the TeaStore application needs to resort to the use of a default image. A first possibility is that service WebUI might trigger the search for another image provider until the connection with the original provider becomes acceptable. Alternatively, WebUI may resort to querying the service Local DB. Or else, if the service Local DB performance is compromised by CPU overload, the WebUI might simply access a predefined file in the local drive, so avoiding further interactions with other services. Obviously, which is the most reasonable options depends on the application.

It is also worth remarking that, depending on the choice of how to realize the adaptation, also the conditions triggering the reconfiguration of the components in charge of the countermeasures to adopt are application-dependent. For instance, the second alternative could depend on the responsiveness of Local DB when the adaptation is required.

Although the actual tasks to execute when a reconfiguration is required are application-dependent, there are activities that are common to all cases. In particular, the detection of anomalous situations that require adaptation and the decision of which countermeasures to take can be cast in the pattern presented in this paper. Indeed, we explore the possibility of reusing known mechanisms borrowed from the *service-oriented computing* (SOC) paradigm.[5] Our exploration hinges on a platform, dubbed SEArch (after Service Execution Architecture) [3], for the discovery and binding of distributed services featuring (non-)functional and behavioural compliance of services. We showcase how our

[5] Although the terminology has evolved, SOC is still widely used and, we believe, still applies to variants such as cloud computing, fog and edge computing, or the many forms of distributed computing associated with what is known as the *Internet of (Every)Things*.

platform can support architectural reconfigurations dictated by changes in the underlying *execution context*. More precisely, in this paper we disregard adaptation that dependents on application-level conditions and focus on reconfigurations due to variations at lower levels such as network speed, CPU load, memory consumption, etc. As we will see, on the one hand this yields a simplified programming pattern for adaptation and, on the other hand, is a limitation of our approach.

Structure of the paper. Section 2 illustrates the key features of our platform necessary to appreciate our programming pattern; more precisely, we survey the behavioural contracts, and the searching and binding mechanism featured by our platform.

Section 3 introduces our programming pattern and how it is cast in the architecture of our platform. We give the architecture of our adaptation framework in Section 3.1; the elements of the programming pattern are key to the design of the control loop that decides how to reconfigure an application at run-time. This decision procedure is detailed in Section 3.2 together with the specification of the adaptation contracts of our running example which concern control data of the execution environment.

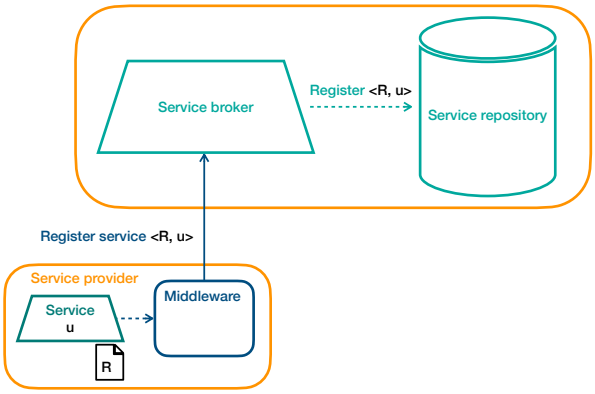
Section 4 describes the adaptation of our running example. We give the behavioural contracts in terms of CFSMs for some scenarios borrowed from the adaptable TeaStore benchmark [2]. Here, we also spell out how the adaptation process is triggered by the control loop given in Section 3.2.

Finally, we draw some conclusions and discuss further work in Section 5.

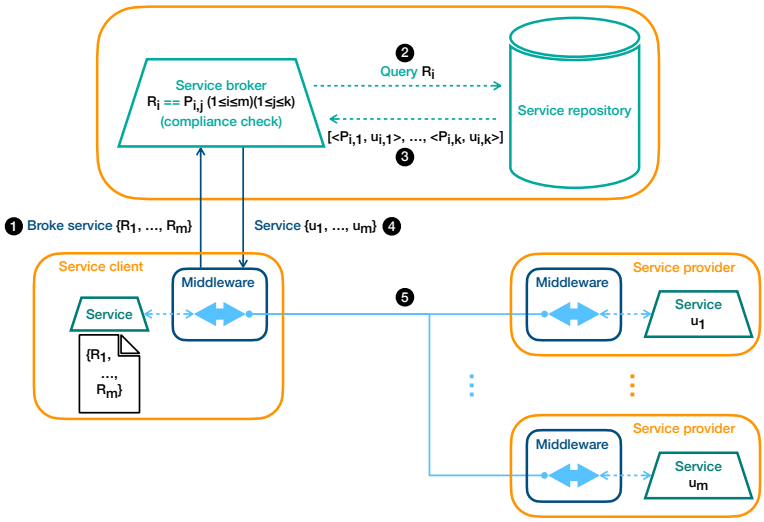
2 An Overview of SEArch

The key role in our platform is played by *service brokers* that are used by *service providers* to register their services, and by *service clients* to dynamically *discover* and *bind* to services satisfying their requirements. These roles are typically advocated in SOAs; the originality of our platform however is the use of *behavioural contracts* to support *semantic-based* composition of services. More precisely, the composition mechanism realised by service brokers of SEArch relies on a notion of behavioural compliance of contracts which we briefly review here.

To support SOC, our platform offers a mechanism for populating service repositories and composing service-based applications. The registration of a service (see Fig. 1a) is conceptually simple: the service provider sends the contract and the unique resource identifier of the provided service, to broker, through its local SEArch middleware. The execution process is significantly more complex since its underlying computational model are *Asynchronous Relational Networks* (ARNs) [4]. This theory envisages *processes* and *communication channels* exposing *ports* with behavioural contracts: processes' ports, called *provides-points*, can be dynamically "coalesed" with ports of communication channels, called *requires-points*, provided that the behavioural contracts associated with



(a) Service registration procedure



(b) Service brokering procedure

Fig. 1: Service management procedures in SEArch

merged ports are "semantically compliant". The notion of compliance adopted is defined in terms of contracts formalised in terms of a bisimilarity relation between *extended Communicating Finite State Machines* (extended CFSM for short) [5], a variant of *communicating finite-state machines* [6] enabling the specification of functional and non-functional behaviour. We now describe our execution process.

When launched on a SEArch node, a service registers the extended CFSMs of its exposed requirement contracts on the local *middleware*, which will mediate

the communication with other services. Basically, the middleware, intercepts the communications a local service attempts over a communication channel, establishes a communication session for that channel (in case the session does not yet exists), and triggers the dynamic reconfiguration sketched in Fig. 1b which we now describe:

❶ the middleware sends the service broker the contract $\{\mathsf{R}_1, \ldots, \mathsf{R}_\mathsf{m}\}$ of the communication channel, consisting of one extended CFSM for each participant required by the service client;
❷ for each extended CFSM R_i in the contract, the service broker queries the service repository for candidates;
❸ the service repository returns a set of candidates[6] $\{\langle \mathsf{P}_{\mathsf{i},1}, \mathsf{u}_{\mathsf{i},1}\rangle, \ldots, \langle \mathsf{P}_{\mathsf{i},\mathsf{k}_\mathsf{i}}, \mathsf{u}_{\mathsf{i},\mathsf{k}_\mathsf{i}}\rangle\}$ where, for all $1 \leq j \leq k_i$, $\mathsf{P}_{\mathsf{i},\mathsf{j}}$ is an extended CFSM and $\mathsf{u}_{\mathsf{i},\mathsf{j}}$ is the URI of the service;
❹ the broker checks whether there is a provision contract $\langle \mathsf{P}_{\mathsf{i},\mathsf{j}}, \mathsf{u}_{\mathsf{i},\mathsf{j}}\rangle$, with $1 \leq j \leq k$, compatible with the requirement contract R_i, for all $1 \leq i \leq m$; this is done by resorting to a bisimilarity check; once the service broker has found services satisfying all the requirement contracts, it returns the set of URIs $\{\mathsf{u}_1, \ldots, \mathsf{u}_\mathsf{m}\}$ to the middleware;
❺ finally, the middleware opens a communication with the middleware of each service returned by the provider requiring the execution of the corresponding service.

Once the composition is achieved, the middleware interacts with the middlewares of the partner services; and the execution process proceeds. Notice that during the execution new requirements might crop up; for instance, because some brokered services need further services.[7] This will initiate a new brokerage phase to discover the newly required services.

The schematic view discussed before establishes several requisites over the implementation of the middleware and the service broker which are omitted as inessential here; interested readers are referred to [3] for details.

3 Adaptability as a programming pattern

In a quest for a notion of adaptation, the authors of [7] advocate a conceptual framework grounded on a neat separation of concerns. Starting from the observation that the behaviour of a component is determined by a program defining *control* and manipulating *data*, adaptation requires to consider, and possibly, modify specific computational or physical resources. The conceptual framework in [7] envisages *adaptation* as the possibility of a component to manipulate and take decisions according to a well-identified set of *control data*. For instance, an

[6] The query is resolved by filtering the extended CFSMs in the repository according to heuristics based on structure of R_i.

[7] This implies that a service may have one or more required-points associated to communication channels of its own.

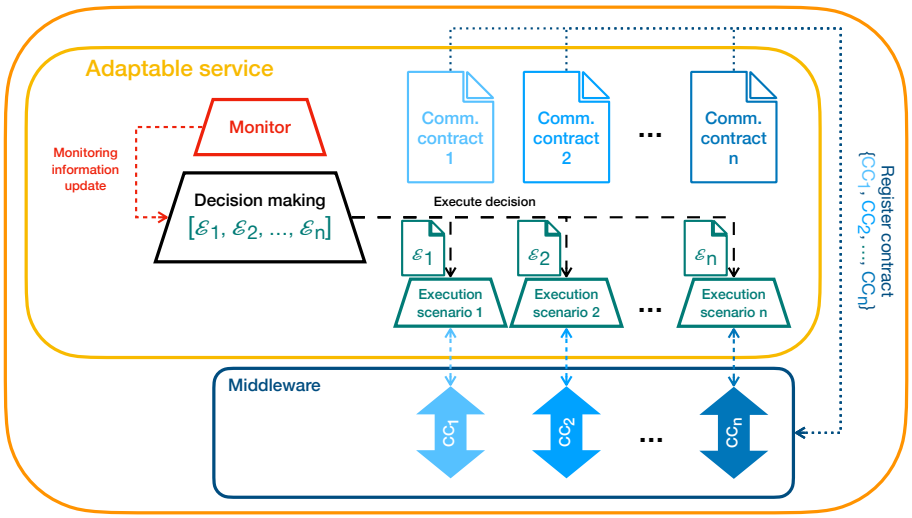


Fig. 2: Architectural view of our programming pattern for adaptation

adaptive behaviour may need to account for memory usage, network traffic, or environmental physical conditions like e.g., temperature.

We render the conceptual framework in our platform as detailed in Sections 3.1 and 3.2.

3.1 An architecture for runtime adaptation

Embracing the conceptual framework introduced in [7] allows one to define adaptation as the property of a component of controlling and possibly modifying at run-time its behaviour based on a specific set of control data.

This acceptation implicitly requires a *monitor* that can collect and make available the control data necessary for enforcing adaptive behaviour. Noteworthy, the nature of control data impacts on the complexity of the monitoring activities. When control data are *exogenous*, namely they originate outside the application (e.g., from sensors, operating system, etc.), their usage in the adaption process can be simply attained by requesting the information to the monitor (e.g., with a request-response interaction). Otherwise, for *endogenous* control data, namely data produced by some components of the application, monitor and application are more coupled and their interactions can require the definition of specific application-level protocols.

Leveraging the API of SEArch, we propose a programming pattern to support the conceptual framework proposed in [7]. We focus on exogenous control data, leaving adaptation endogenous control data for future work.

Fig. 2 provides a high-level architectural view of an adaptable agent implemented over our platform; the two main elements of the architecture are the

adaptable service itself and the local *middleware* (dark blue box) that sets up the communication infrastructure necessary for the dynamic composition of services. As shown, the adaptable service runs behind the middleware, which mediates the interactions of the services/applications in execution on a node with every other component in the network, including other middlewares and the service broker. The mediation basically connects (via the channels $\mathsf{CC}_1, \cdots, \mathsf{CC}_n$) the required ports of the adaptable service with the provided ports of brokered external components.

The structure of an adaptable service is rather complex and consists of:

- a collection of contracts (specified as extended CFSMs) corresponding to each of the execution scenarios ($\{\mathsf{Communication\ contract\ i}\}_{1 \leq i \leq n}$ in the topmost part of the orange box, so that each Communication contract i corresponds to the middleware's channel CC_i; we also emphasise this correspondence with colours),
- a collection of implementations corresponding to each of the execution scenarios $\{\mathsf{Execution\ scenario\ i}\}_{1 \leq i \leq n}$, each equipped with an *adaptation contract* $\mathcal{E}_i$ formalising the environmental conditions under which the execution scenario should be run (bottom-most part of the inner orange box), and
- a decision procedure that runs a control loop, which uses the control data received from the monitor to determine an appropriate execution scenario to run (leftmost part of the orange box).

We remark that each channel CC_i is used by the middleware to trigger the adaptation scenarios according to the Communication contract i. The details of the control loop and of the adaptation contracts of the execution scenarios are given in Section 3.2.

3.2 An implementation of the control loop

As discussed above, taking inspiration from [7], we understand adaptability as the capacity of an agent to respond to changes in the environment. Thus, a significant aspect of our approach relies on the exogenous control data received from the monitor, and how the decision making process uses it to determine an appropriate execution scenario at runtime.

Our programming pattern relies on a fixed set $\mathcal{A}$ of (*quantitative*) *attributes* (ranged over by a) which store the exogenous control data.

Example 1. For the sake of our running example, we make the following assumptions:

- Local DB and WebUI run on the same node while the image provider Image provider runs on a different node (cf. Fig. 4 and Fig. 5 in Section 4);
- adaptations are triggered depending on some constraints involving the attributes L (network latency), R (responsiveness of Image provider), and C (CPU load at the node of WebUI).

The adaptation contracts of our running examples will be constraints on these attributes. ◇

The programming pattern proposed here essentially hinges on SEArch features to instantiate communication channels used in the control loop to manage adaptation. More precisely,

- the invocation of a service triggers the registration of the n alternative communication channels in the middleware of the local node of the service,
- the control loop managing the runtime adaptations consists of iterating the following steps:
  a. acquire from the most recent control data from the monitor (cf. [7]) as an assignment of values to the attributes in $\mathcal{A}$;
  b. select a suitable execution scenario according to the control data provided by the monitor;
  c. trigger the execution of the code implementing the selected scenario;
  d. stop the running execution scenario if the updated control data determine that the current execution scenario is no longer admissible (i.e., the contract of the running scenario is invalidated by newly acquired control data) and repeat from step b.

The control loop sketched above is designed to select one of the possible execution scenarios according to the control data provided by the monitor. More precisely, the identification of a suitable scenario depends on the control data returned by the monitor, which we render as mappings $m : \mathcal{A} \to \Re$ that assign a real value to each attribute in $\mathcal{A}$. This design decision is supported by the framework introduced in [8,9] which guaranties the existence of a decision procedure for determining satisfiability of a class of constraints expressed in a first-order logic with equality whose formulae predicate on real values.[8] Hence, this framework enables the use of *Satisfiability Modulo Theories solvers* [11] (SMT solvers for short) such as Z3 [12], Yices [13], or CVC5 [14]. In fact, we formalise adaptation contracts as constraints on real values.

Example 2. We set the following conditions[9] on L, R, and C:

$$\begin{aligned}
\mathcal{E}_1 &= L < 5ms \quad \wedge \quad R < 1s \\
\mathcal{E}_2 &= \neg\mathcal{E}_1 \quad \wedge \quad 0 \leq C \leq .9 \\
\mathcal{E}_3 &= \neg\mathcal{E}_1 \quad \wedge \quad C > .9
\end{aligned}$$

[8] The framework is based on *real closed fields* [10], a decidable first-order theory of the real numbers [10, Theorem 37]) to specify contracts on quantitative aspects of software artefacts. For the sake of this paper, it is enough to assume that contracts consist of decidable constraints; the interested reader is referred to [8,9] and references therein for details.

[9] The adaptation contracts in our running example are pretty basic, for simplicity. We remark that our platform can handle non-trivial constraints expressed as formulae of the decidable first-order logic over real closed fields.

The intuition behind these conditions is that the interaction between W and P is satisfactory when $\mathcal{E}_1$ holds otherwise an adaptation should take place. We conceive two adaptations when the remote interaction degrades; W retrieves the images through S when the local CPU is not overloaded ($\mathcal{E}_2$), otherwise a default image is displayed ($\mathcal{E}_3$). $\diamond$

The dynamic selection of execution scenarios appeals to the use of SMT solvers to decide the satisfiability of the adaptation contracts associated to the execution scenarios. To tackle this issue, we equip our platform with the procedure expressed in Pythonic pseudo-code in Listing 1.1 which relies on threads and on the following assumptions on the execution scenarios (cf. lines 1-2 of Listing 1.1):

scenarios the functions implementing each execution scenario are collected in a module called ExcScenarios which also exports the list ES of such functions; for simplicity, we assume that these functions do not take parameters;

coordination the module ExcScenarios also exports a boolean variable done, shared with the control loop, and used to determine when the scenario has completed its execution;

termination each execution scenario is guaranteed to atomically set[10] done to True before returning and to terminate.

The control loop starts by setting[11] the done flag to False and entering the loop on line 6 which, until the done flag is not set:

a. chooses a scenario to run if none is running (lines 6-17) and
b. keeps checking if a running scenario should be stopped or it gracefully finished (lines 19-26).

To select the execution scenario, the control loop iterates over the list of function ES and stops as soon as it finds a function whose adaptation contract is true. This is checked by querying an SMT solver whether the negation of $\mathcal{E}_i[a \mapsto \mathsf{m}(a)]_{a \in \mathcal{A}}$ is satisfiable or not;[12]

- gets the control data from the monitor (line 11; assume that monitor_update takes care of communicating with the monitor and returning the mapping to the control loop),
- if the adaptation contract $\mathcal{E}_i$ is violated (namely its negation is satisfiable) under the returned control data, then the scenario cannot be chosen,
- otherwise, the scenario is chosen and a thread running the associated implementation is started (lines 14-15) and the execution continues on line 19.

[10] We use the syntax $\langle \ldots \rangle$ to mark critical sections.

[11] Notice that there is no need to make an atomic assignment since the control loop has not spawned any other thread yet.

[12] As usual, $\mathcal{E}[a \mapsto v_a]_{a \in \mathcal{A}}$ denotes the application to the formula $\mathcal{E}$ of the substitution $[a \mapsto v_a]_{a \in \mathcal{A}}$ which maps each attribute $a \in \mathcal{A}$ to a value v_a.

```
1  from threading import Thread
2  from ExcScenarios import ES, done
3
4  done = False
5
6  while ⟨ not done ⟩:                                  # atomic access
7    stop, scenario = False, None
8    i = 0
9    while scenario == None:
10     # get monitored data
11     m = monitor_update()
12     if not SMT(not ℰ_i[a ↦ m(a)]_{a∈𝒜}) == SAT:
13       # spawn the chosen scenario ES[i]
14       scenario = Thread(target = ES[i])
15       scenario.start()
16     else:
17       i = (i + 1) mod len(ES)
18
19   while not (done or stop):
20     old_m = m
21     # get monitored data
22     m = monitor_update()
23     if not old_m == m:
24       stop = SMT(not ℰ_i[a ↦ m(a)]_{a∈𝒜}) == SAT
25     if stop:
26       scenario.terminate()
```

Listing 1.1: Pythonic pseudo-code implementing the control loop.

– the loop on lines 19-26 keeps checking that the adaptation contract of the running scenario is still valid; if the contract gets violated, the scenario is terminated (line 26).

For simplicity, the thread running the execution scenario is terminated; we indeed opted for a lighter presentation instead of devising a more general mechanism to coordinate the thread and the control loop for a graceful termination (where e.g., the thread restores some application-dependent invariants before terminating) at the cost of complicating the presentation.

4 The Adaptable TeaStore in SEArch

We now apply the programming pattern presented in Section 3 to our running example and detail how the control loop of Listing 1.1 manages the reconfigurations specified by the policies established with the adaptation contracts in Example 2.

4.1 Deploying our running example

Our control loop can be thought as a scheme for a "combinator" that allows to build adaptable services from other services. One such a combinator is provided, the execution of an adaptable service would be as follows:

1. the communication with the local middleware is established and the communication contracts are registered in the middleware;
2. the service is connected to the communication channels created by the middleware for its execution scenarios and the decision making procedure executing our control loop is spawned;
3. the decisions making procedure requests the monitor for an update on the adaptation data and chooses the first adaptation contract satisfied by it (lines 6-17 of Listing 1.1).

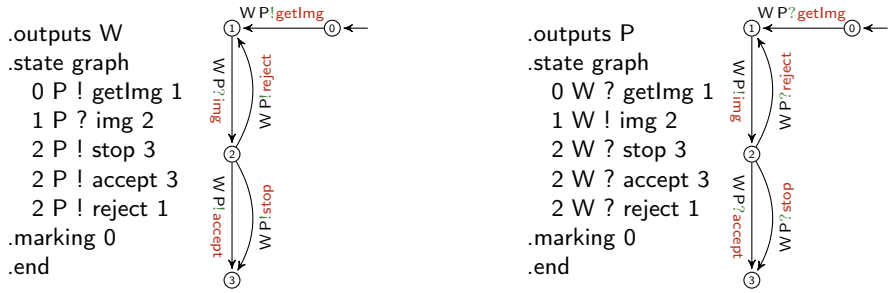
While the selected execution scenario runs, the control loop continues by checking if an update of the control data violates the adaptation contract of the running execution scenario. If such a violation occurs, the running execution scenario is terminated and the decision making procedure repeats step 3 above in order to adapt the runtime behaviour of the adaptable service to the new environmental conditions.

We instantiate the procedure above to deploy an adaptable service WebUI of our running example using the communication contracts in Fig. 3, which yields both the textual format of CFSMs (which SEArch borrows from [15]) and their usual graphical notation where initial states are marked by an incoming arrow with no source state and transitions are labelled with send or receive actions (e.g., W S!query represents the sending of message query done by W to S and W S?query is the dual receive action).

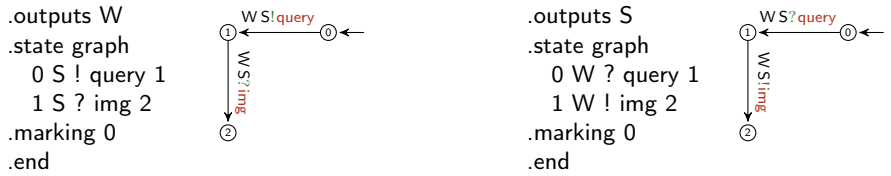
We start by observing that the communication contract corresponding to $\mathcal{E}_3$ in Fig. 3c consists just of a CFSM with a single state and no transitions since the adaptation does not involve any interactions (e.g., the default image can be read from a file on a local hard drive). The contract involving the persistent service (Fig. 3b) consists of the CFSM S used to query the broker for a storage service that can respond to a query from WebUI by sending an image. The CFSM W in (Fig. 3b) specifies the behaviour of WebUI in this scenario. Likewise, the CFSM W in Fig. 3a specifies how WebUI interacts with an external image provider queried to the broker using the CFSM P which features a more complex protocol to allow WebUI to repeatedly ask for images when those provided by the external service are not suitable, or to stop the interaction with P.

The architectural view of the scenario involving the external image provider is depicted in Fig. 4. This view casts the architecture in Fig. 2 for the running example. More precisely, the WebUI:

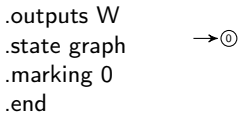
1. registers the Image provider contract, used by the local process Image provider proxy to communicate with an external image provider service (see Fig. 3a),



(a) Contract for external service



(b) Contract for the fallback scenario with the storage service



(c) Contract for the use of default locally stored images

Fig. 3: Contracts for the scenario-based adaptable TeaStore

the Database contract, used by the local process Local DB proxy to communicate with a locally hosted database service Local DB (see Fig. 3b), and Default image contract, used by the local process Default image[13] (see Fig. 3c), in the Middleware

2. initiates the execution of the Monitor, and
3. spawns the decision making procedure executing the control loop, which asks the monitor for the first update and chooses one of the execution scenarios accordingly.

Let us now consider how the architecture can possibly evolve at runtime when control data changes.

[13] Recall that this contract yields a communication channel whose behaviour reflects that no communication will take place.

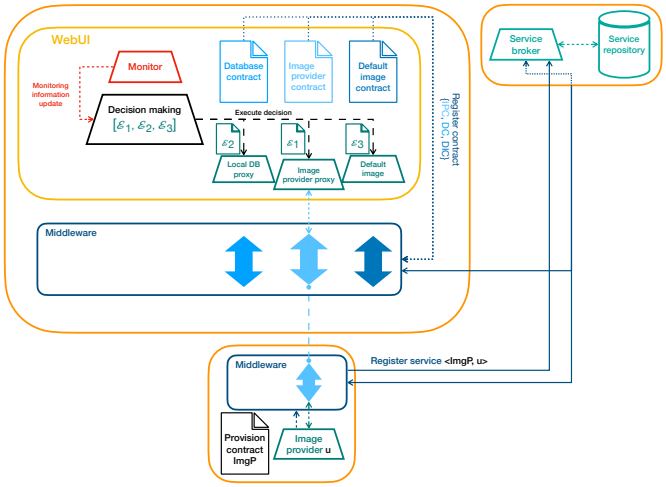


Fig. 4: Architectural view for the external service scenario

4.2 No degradation of the interaction with the image provider

As long as runtime changes of control data do not invalidate the adaptation contract $\mathcal{E}_1$, the architecture stays as in Fig. 4; in fact,

Step 1 the Monitor responds to monitor_update() with $\{L \mapsto 4.7, R \mapsto 0.6, C \mapsto 0.54\}$ (line 11);

Step 2 not $\mathcal{E}_1[L \mapsto 4.7, R \mapsto 0.6, C \mapsto 0.54]$ is not SAT (i.e., UNSAT) thus, the control loop creates a thread for executing the local process Image provider proxy, connected to its associated communication channel generated from the contract Image provider contract (lines 12-15), yielding the architecture in Fig. 4;

Step 3 in this thread, the local process Image provider proxy implements the communication pattern of the CFSM W in the contract of Fig. 3a; in particular, it starts by sending the message getImg thus triggering the brokering process in the Service broker (see Fig. 1b and the corresponding reference)

Step 4 the external service Image provider (executing the process Image provider) is bound to the application and, since it implements a communication pattern compliant to the CFSM P in Fig. 3a, it receives the request and returns an image to the caller by sending a message img;

Step 5 as the thread executing the local process Image provider proxy has not set the shared variable done and there is no instruction to stop the execution through the variable stop, the invocation to monitor_update() obtains an update of the control data (line 22) from the Monitor.

We now consider cases where the environmental conditions violate the adaptation contract $\mathcal{E}_1$ of the running execution scenario.

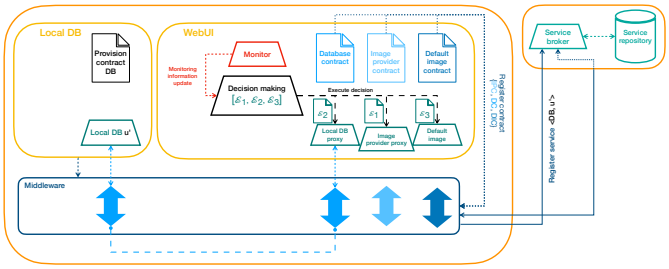


Fig. 5: Reconfiguration according to adaptation contract $\mathcal{E}_2$ upon violation of $\mathcal{E}_1$

4.3 Degradation of the interaction with the image provider

In our running example, a degradation of the interaction between WebUI (more specifically, the local process Image provide proxy) and the external service Image provider requires a reconfiguration when the latency (L) or the responsiveness (R) becomes too high. Let us assume that the external image provider service becomes unresponsive, the other case is similar.

Step 6 while the local process Image provide proxy interacts with the service Image provider through the communication channel generated from the Image provider contract, the Monitor responds to monitor_update() with $\{L \mapsto 5.3, R \mapsto 0.7, C \mapsto 0.51\}$ (line 22);

Step 7 the control loop accounts for the change in m and, as not $\mathcal{E}_1[L \mapsto 5.3, R \mapsto 0.7, C \mapsto 0.51]$ now is SAT because $L \geq 5ms$, terminates the thread executing the local process Image provider proxy using terminate() and returns to the code fragment that performs the selection of an appropriate execution scenario (lines 9-17);

Step 8 not $\mathcal{E}_2[L \mapsto 5.3, R \mapsto 0.7, C \mapsto 0.51]$ is not SAT thus, the control loop creates a thread for executing the local process Local DB proxy, connected to its associated communication channel generated from the contract Database contract (lines 12-15), yielding the architecture in Fig. 5;

Step 9 in this thread, the local process Local DB proxy implements the communication pattern of the CFSM W in the contract of Fig. 3b; in particular, it starts by sending the message query thus triggering the brokering process in the Service broker but fixing the local service Local DB as the provider;

Step 10 the service Local DB (executing the process Local DB) is bound to the application and, since it implements a communication pattern compliant to the CFSM S in Fig. 3b, it receives the request and returns an image to the caller by sending a message img.

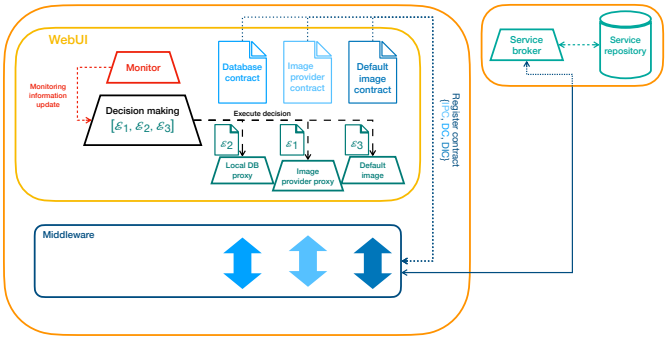


Fig. 6: Reconfiguration according to adaptation contract $\mathcal{E}_3$ upon violation of $\mathcal{E}_2$

4.4 Local node overloaded

According to our adaptation contracts, the execution shown in Section 4.3 requires a reconfiguration when the CPU gets overloaded. Our control loop will therefore terminate the thread running the process Local DB proxy.

Step 11 while the local process Local DB proxy interacts with the service Local DB through the communication channel generated from the contract Database contract, the Monitor responds to the invocation of monitor_update() with $\{L \mapsto 6.2, R \mapsto 0.4, C \mapsto 0.92\}$ (line 22);

Step 12 the control loop accounts for the change in m and, as not $\mathcal{E}_2[L \mapsto 6.2, R \mapsto 0.4, C \mapsto 0.92]$ now is SAT because $C \geq 0.9$, terminates the thread executing the local process Local DB proxy using terminate() and returns to the code fragment that performs the selection of an appropriate execution scenario (lines 9-17);

Step 13 not $\mathcal{E}_3[L \mapsto 6.2, R \mapsto 0.4, C \mapsto 0.92]$ is not SAT thus, the control loop creates a thread for executing the local process Default image, yielding the architecture in Fig. 6 (lines 12-15);

Step 14 the execution continues according to the behaviour of the local process Default image. This process is implemented satisfying Default image contract so no communication takes place over the generated communication channel, consequently, the task of finding an image is performed resorting only to resources that are locally accessible by the local process Default image.

5 Conclusions

In this paper we advocate a programming pattern for adaptations that possibly reconfigure the architecture of service-oriented applications. Our pattern is supported by SEArch, a platform featuring semantic-based dynamic discovery and binding of services. We identified some reconfigurations policies on the adaptable TeaStore benchmark to demonstrate our approach; in particular, our policies fit

with the scenarios in [2, Sections 3.2.3 and 3.2.4]. This benchmark includes other scenarios that we did not consider here; however, we argue that our approach is general enough to express the other scenarios specified in [2] since the architectural reconfigurations they encompass depend only on exogenous control data.

As recalled in Section 2, SEArch contracts are formalised in terms of extended CFSMs. This allows us to specify contracts concerning application-level (non-)functional requirements. For instance, the quality of the image hinted in our running example could be modelled as a quantitative attribute, say Q. As mentioned in Section 3 we assume that control data are exogenous. This implies that they cannot include application-level information; for instance, Q cannot be used in the control loop to trigger an adaptation when the image provided is sub-optimal (in fact, our solution in Section 4 handles this in the application contract of Fig. 3a and not in adaptation contracts). Overcoming this limitation requires the introduction of mechanisms to specify the interactions among the monitor and the execution scenarios. This can be readily supported in SEArch since the monitor can be conceived as a service local to a node whose behaviour is specified by a suitable extended CFSM. Even further, our platform could also support monitors as outsourced services that could be dynamically discovered and used by other services once suitable behavioural contracts are established for their usage. These generalisations are in scope of future work.

Our programming pattern hinges on the control loop presented in Section 3 that, liaising with a monitor that observes control data, decides if a run-time architectural reconfiguration is necessary and triggers the application-dependent activities provided by the developer. One could simplify this architecture by embedding the control loop in the monitor. We opted for a solution with two separated components because it yields more flexiblility and can be extended more easily. For instance, the generalisation to endogenous control data can be attained more straightforwardly in our solution that in one where control loop and monitor are entangled.

Our control loop can –at an abstract level– be envisaged as "combinator" that composes the application-dependent functions (the adaptation scenarios) necessary for reconfigurations. In this paper, we adopted a rather simplistic mechanism for the composition of these functions. Depending on the nature of the reconfigurations required by an application, it could be necessary to appeal to more sophisticated composition mechanisms. For instance, it could be necessary that the control loop passes some data when invoking the adaptation scenario. This can be supported by our platform at the cost of more complex interactions between the control loop and execution scenarios.

Finally, we recall that, for simplicity, the running adaptation scenario is abruptly terminated when a reconfiguration is triggered (line 26 in Listing 1.1). This may not be desirable; we plan to develop mechanisms for guaranteeing graceful termination in future work.

Acknowledgements. We thank reviewers for their thorough reports and their suggestions that helped us to improve the paper.

References

1. von Kistowski, J., Eismann, S., Schmitt, N., Bauer, A., Grohmann, J., Kounev, S.: Teastore: A micro-service reference application for benchmarking, modeling and resource management research. In: 26th IEEE International Symposium on Modeling, Analysis, and Simulation of Computer and Telecommunication Systems (MASCOTS 2018), IEEE Computer Society (September 2018) 223–236
2. Bliudze, S., de Palma, G., Giallorenzo, S., Lanese, I., Zavattaro, G., Ndadji, B.A.Z.: Adaptable TeaStore. On-line (2024) Available at https://arxiv.org/abs/2412.16060.
3. Lopez Pombo, C.G., Montepagano, P., Tuosto, E.: SEArch: an execution infrastructure for service-based software systems. In Castellani, I., Tiezzi, F., eds.: Proceedings of Coordination Models and Languages - 26nd IFIP WG 6.1 International Conference, COORDINATION 2024, Held as Part of the 19th International Federated Conference on Distributed Computing Techniques, DisCoTec 2024. Volume 14676 of Lecture Notes in Computer Science., Springer-Verlag (June 2024) 314–330
4. Fiadeiro, J.L., Lopes, A.: An interface theory for service-oriented design. Theoretical Computer Science **503** (2013) 1–30
5. Lopez Pombo, C.G., Martinez Suñé, A.E., Melgratti, H.C., Senarruzza Anabia, D.N., Tuosto, E.: Behavioural, functional, and non-functional contracts for dynamic selection of services. In Giusto, C.D., Ravara, A., eds.: Proceedings of Coordination Models and Languages - 27nd IFIP WG 6.1 International Conference, COORDINATION 2025, Held as Part of the 20th International Federated Conference on Distributed Computing Techniques, DisCoTec 2025. Volume 15731 of Lecture Notes in Computer Science., Springer-Verlag (June 2025) 153–174
6. Brand, D., Zafiropulo, P.: On communicating finite-state machines. Journal of the ACM **30**(2) (1983) 323–342
7. Bruni, R., Corradini, A., Gadducci, F., Lluch Lafuente, A., Vandin, A.: A conceptual framework for adaptation. In de Lara, J., Zisman, A., eds.: Fundamental Approaches to Software Engineering, Berlin, Heidelberg, Springer Berlin Heidelberg (2012) 240–254
8. Lopez Pombo, C.G., Martinez Suñé, A.E., Tuosto, E.: A dynamic temporal logic for quality of service in choreographic models. Theoretical Computer Science **1043** (2025) 115247
9. Lopez Pombo, C.G., Martinez Suñé, A.E., Tuosto, E.: A dynamic temporal logic for quality of service in choreographic models. In Ábrahám, E., Dubslaff, C., Tarifa, S.L.T., eds.: Proceedings of 20th International Colloquium on Theoretical Aspects of Computing - ICTAC 2023. Volume 14446 of Lecture Notes in Computer Science., Lima, Perú, Springer-Verlag (December 2023) 119–138
10. Tarski, A.: A Decision Method for Elementary Algebra and Geometry. University of California Press (1951) Originally published in [16] and reprinted in [17].
11. Biere, A., Heule, M., Van Maaren, H., Walsh, T., eds.: Handbook of Satisfiability: Second Edition. Volume 336 of Frontiers in Artificial Intelligence and Applications. IOS Press (February 2021)
12. de Moura, L.M., Bjørner, N.: Z3: an efficient SMT solver. In Ramakrishnan, C., Rehof, J., eds.: Proceedings of 14th International Conference Tools and Algorithms for the Construction and Analysis of Systems TACAS 2008, Held as Part of the Joint European Conferences on Theory and Practice of Software, ETAPS 2008. Volume 4963 of Lecture Notes in Computer Science., Springer-Verlag (March 2008) 337–340

13. Dutertre, B.: Yices 2.2. In Biere, A., Bloem, R., eds.: Proceedings of the 26th International Conference Computer Aided Verification CAV 2014, Held as Part of the Vienna Summer of Logic, VSL 2014. Volume 8559 of Lecture Notes in Computer Science., Springer-Verlag (July 2014)
14. Barbosa, H., Barrett, C.W., Brain, M., Kremer, G., Lachnitt, H., Mann, M., Mohamed, A., Mohamed, M., Niemetz, A., Nötzli, A., Ozdemir, A., Preiner, M., Reynolds, A., Sheng, Y., Tinelli, C., Zohar, Y.: cvc5: A versatile and industrial-strength SMT solver. In Fisman, D., Roşu, G., eds.: Proceedings of the 28th International Conference Tools and Algorithms for the Construction and Analysis of Systems TACAS 2022, Held as Part of the European Joint Conferences on Theory and Practice of Software, ETAPS 2022 Part I. Volume 13243 of Lecture Notes in Computer Science., Springer-Verlag (April 2022) 415–442
15. Coto, A., Guanciale, R., Lange, J., Tuosto, E.: ChorGram: tool support for choreographic development. Available at https://bitbucket.org/eMgssi/chorgram/src/master/ (2015)
16. Tarski, A.: A decision method for elementary algebra and geometry. Memorandum RM-109, RAND Corporation (1951) Later published in [10] and reprinted in [17].
17. Tarski, A. Texts and Monographs in Symbolic Computation. In: A Decision Method for Elementary Algebra and Geometry. Springer-Verlag (1998) 24–84 Originally published in [16] and reprinted from [10].